

A Project Report on
“FEE MANAGEMENT SYSTEM”
MASTER OF COMPUTER APPLICATION

BY
KUMARI SAPNA GORAI
Enrollment No.-AJU/210019

Under the esteemed guidance of
DR. ARVIND KUMAR PANDEY
ALKA KUMARI
(Internal Guide)



DEPARTMENT OF COMPUTER SCIENCE & IT
ARKA JAIN UNIVERSITY, JHARKHAND
Jamshedpur
2021-2023

ARKA JAIN UNIVERSITY

The image shows the main entrance gate of Arka Jain University. The gate is a large, white, rectangular structure with a flat roof. The words "ARKA JAIN UNIVERSITY" are written in large, black, capital letters across the top of the gate. The gate is flanked by two white pillars. Between the pillars is a large, dark brown wooden gate. A person in a blue shirt and dark pants is standing near the gate. The gate is set against a background of green trees and a clear blue sky. The ground in front of the gate is paved with light-colored stones. There are some potted plants and small trees on either side of the gate.

GUIDED BY

DR. ARVIND PANDEY
ALKA KUMARI
(Internal Guide)

SUBMITTED BY

KUMARI SAPNA GORAI

SUBMITTED TO

DEPARTMENT OF COMPUTER SCIENCE & IT
ARKA JAIN UNIVERSITY

A Project Report on
FEE MANAGEMENT SYSTEM

Submitted in partial fulfilment
for the award of the Degree in

MASTER OF COMPUTER APPLICATION

By

Kumari Sapna Gorai

Enrollment No. - AJU/210019

Under the guidance of
Dr. Arvind kumar Pandey

Alka Kumari
(Internal Guide)



ARKA JAIN UNIVERSITY

JAMSHEDPUR, JHARKHAND

DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY
2021-23

ARKA JAIN UNIVERSITY, JHARKHAND

DEPARTMENT OF COMPUTER SCIENCE & IT



CERTIFICATE

This is to certify that the project entitled, "**Fee Management System**", is bonafied work of **Kumari Sapna gorai** bearing **Enrolment no- AJU/210019** submitted in partial fulfilment of the requirements for the award of degree of Master OF COMPUTER APPLICATIONS (MCA) from ARKA JAIN University, JHARKHAND.

Internal Guide

HOD



University Seal

Date: 23.7.2022

ABSTRACT

A fee management system is application that is designed to manage all the operation related fee of an institution. It helps to maintain the database of new students and the fee. User or admin can see all the data about fee of all the students. The best way to maintain, organize, and handle the large amount data of students and their fee details is fee management system software.

Over all this project of ours is being developed to help the students as well as staff of fee department to maintain the fee in the best way possible and also reduce the human efforts.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to several individuals and organization for supporting me throughout the completion of my project.

First, I wish to express my sincere gratitude to my mentor (Miss. Alka Singh) for his enthusiasm patience, insightful comments, helpful information, practical advices and unceasing ideas that have helped me tremendously at all times in my Project and writing of these thesis. His immense knowledge, profound experience and professional expertise in Backend has enabled me to complete this project successfully. Without his support and guidance, this project would not have been possible.

I am also thankful to our respected H.O.D (Dr. Arvind Kumar Pandey) and all faculty members for loving inspiration and timely guidance. I also wish to express my sincere thanks to the Department of Computer science & Information technology of ARKA JAIN UNIVERSITY for accepting this project.

Thanks for all your encouragement!

TABLE OF CONTENTS

Chapter 1

Introduction

1.1. Overview	1
1.2. Objectives	1
1.3 benefits of new system.....	1
1.4. Identification of Needs	2

Chapter 2

2.1. Visual Studio Code	3
2.2. Languages	3
2.2.1. Html	3
2.2.2. CSS	3
2.2.3. JavaScript	3
2.2.4. PHP	4
2.2.5. MySQL	4

Chapter 3

Requirements and analysis

3.1. Sra (Software Requirement Analysis)	5
3.2. Preliminary Investigation	7
3.3. Feasibility Study.....	8
3.4. System Analysis	9
3.5. Hardware and Software Requirements	10
3.6. DFD	11
3.6.1. Context Level DFD	14
3.6.2. DFD Level 1	15
3.6.3. DFD Level 2	17
3.7. ER-Diagram	21

3.9. Normalization	21
--------------------------	----

3.10. Gantt Chart.....	23
------------------------	----

Chapter 4

4.1. Input to the project	24
---------------------------------	----

4.2. Output to the project	25
----------------------------------	----

4.3. Modularization details	26
-----------------------------------	----

4.4. Data Integrity	27
---------------------------	----

4.5. Data Dictionary	28
----------------------------	----

Chapter 5

Program Code and Testing

5.1. Code Details.....	38
------------------------	----

5.2. Testing Approach	39
-----------------------------	----

5.2. Use Case	40
---------------------	----

5.3. Validation Check	40
-----------------------------	----

Chapter 6

conclusion

6.1. Future Scope	40
-------------------------	----

6.2 Conclusion	40
----------------------	----

6.3. References	41
-----------------------	----

1. INTRODUCTION

A fee management system is application that is designed to manage all the operation related fee of an institution. It helps to maintain the database of new students and the fee. User or admin can see all the data about fee of all the students. The best way to maintain, organize, and handle the large amount data of students and their fee details is fee management system software.

Over all this project of ours is being developed to help the students as well as staff of fee department to maintain the fee in the best way possible and also reduce the human efforts.

1.1. Definition of problem

The problem occurred before having computerized system includes:

- File lost- When computerized system is not implemented file is always lost because of human environment. Sometimes due to some human error there may be a loss of records.
- File damaged -When a computerized system is not their file is always lost due to some accident like spilling of water by some member on file accidentally. Besides some natural disaster like floods or fires may also damage the files.
- Difficult to search record- When there is no computerized system there is always a difficulty in searching of records if the records are large in number.
- Space consuming- After the number of records become large the space for physical storage of file and records also increases if no computerized system is implemented.
- Cost consuming- As there is no computerized system the to add each record paper will be needed which will increase the cost for the management of library.

1.2. Modular Definition

This section gives a brief description of the modules used in the project:

1.2.1. Admin Module

Input – Admin enters the login username and password.

Process Definition – Checks login username and password is valid or not.

Output – Admin login to the Main form and can add students record, new user record generate reports etc.

1.2.2. New User Module

Input – User enter their details to create new account.

Process Definition – Checks whether he entered all details or not.

Output – User can login and manage the fee detail of the students

1.2.3. Student Module

Input – Admin or User will entered the details of student

Process Definition – Checks whether he entered all details or not.

Output – Records will be added to the database.

1.2.4. Course Module

Input – Admin or User will entered the details of the new course

Process Definition – Admin or User will entered the details of the course

Output – Records will be added to the database and it show when new student data will be added at the time of course Selection

1.2.5. Fee Module

Input – Admin or user will take the fee of the student and save the record on database

Process Definition – Checks whether the fee record is clear or not

Output – Record will be added to the database.

2. SYSTEM ANALYSIS

System analysis is the process of gathering and interpreting facts, diagnosing problems and using the information to recommend improvements on the system. System analysis is a problem-solving activity that requires intensive communication between the system users and system developers. System analysis or study is an important phase of any system development process. The system is viewed as a whole, the inputs are identified and the system is subjected to close study to identify the problem areas. The solutions are given as a proposal. The proposal is reviewed on user request and suitable changes are made. This loop ends as soon as the user is satisfied with the proposal.

2.1. Existing System

- In Early days Fees are managed manually. It required lot of time to record or to retrieve the details. The employees who have to record the details must perform their job very carefully. Even a small mistake would create a lot of problems. Security of information is very less. Report generations of all the information is very tough task.
- Maintenance of Fee catalogue and arrangement of the Records to the catalogue is very complex task. In addition to its maintenance of member details, fee dates and fine dates etc. manually is a complex task.
- All the operations must be performed in perfect manner for the maintenance of the fee department without any degradation which may finally result in the failure of the entire system.

2.2. Proposed System

To solve the inconveniences as mentioned in the existing system, an computerize fee management system is proposed. The proposed system contains the following features:

- Admin can logging and maintain his details and can add other user or staff details, students details, can add new course, and take and generate receipt and report of about students fee.

- Individually each member will have his account through which he can access the 4.4

2.3. Feasibility Study

Feasibility requirements are undertaken to have a view of understanding the features that are incorporated in terms of the requirements to design and implementation. The references of the feasibility study will be divided into multiple sections so that the understandability can be organized so for example if we have to design software the required investment and flow of cash has to be properly analysed in the same way technical associations are acquired to be discussed.

- Technical feasibility.
- Operational feasibility.
- Economic feasibility.

2.3.1. Technical feasibility

The customization that has to be operational when a particular identity or field is changed has to be replicated in real time which has to be properly designed and tested for the workability. All types of working references are provided in terms of the resources within the company premises. The integrated settings that are required for the accompanied sharing and alert system will also be associated properly and the vendor integrations will be tested for security provisions. The accessibility of the standards that are required will be properly define so as to acknowledge the formats of reports which has to be in-clubbed in different types of reporting sections that are incorporated.

2.3.2. Operational feasibility

Apprehension of working is required to be properly planed as when the system will be provided to multiple users a proper support has to be provided as multiple integrated functionalities are provided. The applications that may arise has to be drafted it so that in the real time working it can be properly acknowledged as multiple Global finance will be incorporated to the system. A

proper documentation will be provided with all variations of report and the standing in the customization options that are provided under the setting.

2.3.3. Financial feasibility

Financial return on investment calculation so will be done by the financial team with the help of multiple associated software that are available will be used. Considerations of the cash flow will be determined as the source of income has to be properly acknowledged

2.4. Non-Functional Requirements

Non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviours. They are contrasted with functional requirements that define specific behavior or functions

2.4.1. Efficiency requirement

When a fee management system will be implemented staff and user will easily access fee and students records as searching and fee transaction will be very faster.

2.4.2. Reliability requirement

The system should accurately perform member registration, member validation, report generation, fee transaction and search

2.4.3. Usability requirement

The system is designed for a user-friendly environment so that student and staff of fee department can perform the various tasks easily and in an effective way.

2.4.4. Organizational requirement

The organization's vision, goals, objectives and priorities. Business and performance plans. Systems, processes and requirements for quality assurance.

2.4.4.1. Implementation requirements

In implementing whole system it uses Visual Studio .net 2019 as IDE and Use Visual Basic Programming language for front end design and use Microsoft SQL server as back end.

4.4.4.2. Delivery requirements

The whole system is expected to be delivered in six months of time with a weekly evaluation by the project guide.

2.5. Functional requirements

Functional requirement defines a function of a system or its component, where a function is described as a specification of behavior between inputs and outputs.

2.5.1. User login

Description of feature

This feature used by the user to login into system. They are required to enter user id and password before they are allowed to enter the system. The user id and password will be verified and if invalid id is there user is allowed to not enter the system.

Functional requirements

- user id is provided when they register
- The system must only allow user with valid id and password to enter the system

- The system performs authorization process which decides what user level can access to.
- The user must be able to logout after they finished using system.

2.5.2. Register new user

Description of feature

This feature can be performed by all users to register new user to create account.

Functional requirements

- System must be able to verify information
- System must be able to delete information if information is wrong

2.5.3. Search students records

Description of feature

This feature is found in new student's maintenance part. We can search students based on student id, Student name.

2.5.4. ADMIN

2.5.4.1. Manage User

Description of feature

In this feature Admin can add user, delete user and view user.

2.5.4.2. Manage Students Record

Description of feature

Admin can add new Students details, delete students and student's details and can search for a Student.

3. SYSTEM DESIGN

System design is the solution for the creation of a new system. This phase focuses on the detailed implementation of the feasible system. It emphasis on translating design. Specifications to performance specification. System design has two phases of development • Logical design • Physical design During logical design phase the analyst describes inputs (sources), outputs (destinations), databases (data sores) and procedures (data flows) all in a format that meets the user requirements. The analyst also specifies the needs of the user at a level that virtually determines the information flow in and out of the system and the data resources. Here the logical design is done through data flow diagrams and database design. The physical design is followed by physical design or coding. Physical design produces the working system by defining the design specifications, which specify exactly what the candidate system must do. The programmers write the necessary programs that accept input from the user, perform necessary processing on accepted data and produce the required report on a hard copy or display it on the screen.

3.1. Input design

Input design is the link that ties the information system into the world of its users. The input design involves determining the inputs, validating the data, minimizing the data entry and provides a multi-user facility. Inaccurate inputs are the most common cause of errors in data processing. Errors entered by the data entry operators can be controlled by input design. The user-originated inputs are converted to a computer-based format in the input design. Input data are collected and organized into groups of similar data. Once identified, the appropriate input media are selected for processing. All the input data are validated and if any data violates any conditions, the user is warned by a message. If the data satisfies all the conditions, it is transferred to the appropriate tables in the database. In this project the student details are to be entered at the time of registration. A page is designed for this purpose which is user friendly and easy to use. The design is done such that users get appropriate messages when exceptions occur.

3.2. Output Design

Computer output is the most important and direct source of information to the user. Output design is a very important phase since the output needs to be in an efficient manner. Efficient and intelligible output design improves the system relationship with the user and helps in decision making. Allowing the user to view the sample screen is important because the user is the ultimate judge of the quality of output. The output module of this system is the selected notifications.

3.3. Database Design

Databases are the storehouses of data used in the software systems. The data is stored in tables inside the database. Several tables are created for the manipulation of the data for the system.

Two essential settings for a database are-

- The field that is unique for all the record occurrences.
- The field used to set relation between tables.

3.4. System Tools

The various system tools that have been used in developing both the front end and the back end of the project are being discussed in this unit.

Visual studio Code

Visual Studio Code combines the simplicity of a source code editor with powerful developer tooling, like IntelliSense code completion and debugging.

First and foremost, it is an editor that gets out of your way. The delightfully frictionless edit-build-debug cycle means less time fiddling with your environment, and more time executing on your ideas.

Languages

HTML

HTML stands for **Hypertext Markup Language**. It allows the user to create and structure sections, paragraphs, headings, links, and blockquotes for web pages and applications.

HTML is not a programming language, meaning it doesn't have the ability to create dynamic functionality. Instead, it makes it possible to organize and format documents, similarly to Microsoft Word.

CSS

Cascading Style Sheets, fondly referred to as CSS, is a simple design language intended to simplify the process of making web pages presentable.

CSS handles the look and feel part of a web page. Using CSS, you can control the color of the text, the style of fonts, the spacing between paragraphs, how columns are sized and laid out, what background images or colors are used, layout designs, variations in display for different devices and screen sizes as well as a variety of other effects.

JavaScript

JavaScript is a dynamic computer programming language. It is lightweight and most commonly used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages. It is an interpreted programming language with object-oriented capabilities.

Why Using PHP?

There are a lot of reasons to know and love PHP, probably the most potent and valid of which is this: it's used and runs everywhere the web does. Your cheap little \$3 per month hosting account *may* let you run a web application in Python or Ruby if you shop carefully. But it'll definitely run PHP. This means that you can count on it wherever you are.

And because it runs everywhere, and is easy to get started with, *a lot* of very popular software is written in PHP. **WordPress** is the example that's both largest and most familiar to me, but tools

like Joomla, Drupal, Magento, ExpressionEngine, vBulletin (yep, that's still around), MediaWiki, and more are all running PHP on the server.

Why using MYSQL?

Many of the world's largest and fastest-growing organizations including Facebook, Google, Adobe, Alcatel Lucent and Zappos rely on MySQL to save time and money powering their high-volume Web sites, business-critical systems and packaged software.

Since then, the performance & scalability, reliability, and ease of use of the world's most **popular** open source database, characteristics that made **MySQL** the #1 choice for web applications, have relentlessly been improved.

4. Requirement Specification

Hardware Configuration

RAM	1GB
Hard Disk	3GB+
Processor	CORE i3 and above

Software Requirements

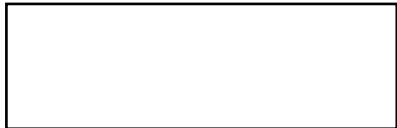
- Operating system: Windows 10
- Front end: Html, CSS, JavaScript
- Frameworks and library: Bootstrap, Font awesome, Google fonts
- Backend: PHP
- Text Editor: Visual Studio Code
- Database: MySQL
- Web Server: Apache

5. DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a structured analysis and design tool that can be used for flowcharting. A DFD is a network that describes the flow of data and the processes that change or transform the data throughout a system. This network is constructed by using a set of symbols that do not imply any physical implementation. It has the purpose of clarifying system requirements and identifying major transformations. So, it is the starting point of the design phase that functionally decomposes the requirements specifications down to the lowest level of detail. DFD can be considered to an abstraction of the logic of an information oriented or a process-oriented system flow-chart. For these reasons DFD's are often referred to as logical data flow diagrams.

EXTERNAL ENTITY

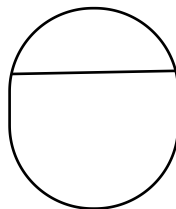
An external entity is a source or destination of a data flow. Only those entities which originate or receive data are represented on a data flow diagram. The symbol used is a rectangular box.



Entity

PROCESS

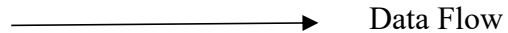
A process shows a transformation or manipulation of data flow within the system. The symbol used is an oval shape.



Process

DATAFLOW

The data flow shows the flow of information from a source to its destination. Data flow is represented by a line, with arrowheads showing the direction of flow. Information always flows to or from a process and may be written, verbal or electronic. Each data flow may be referenced by the processes or data stores at its head and tail, or by a description of its contents.

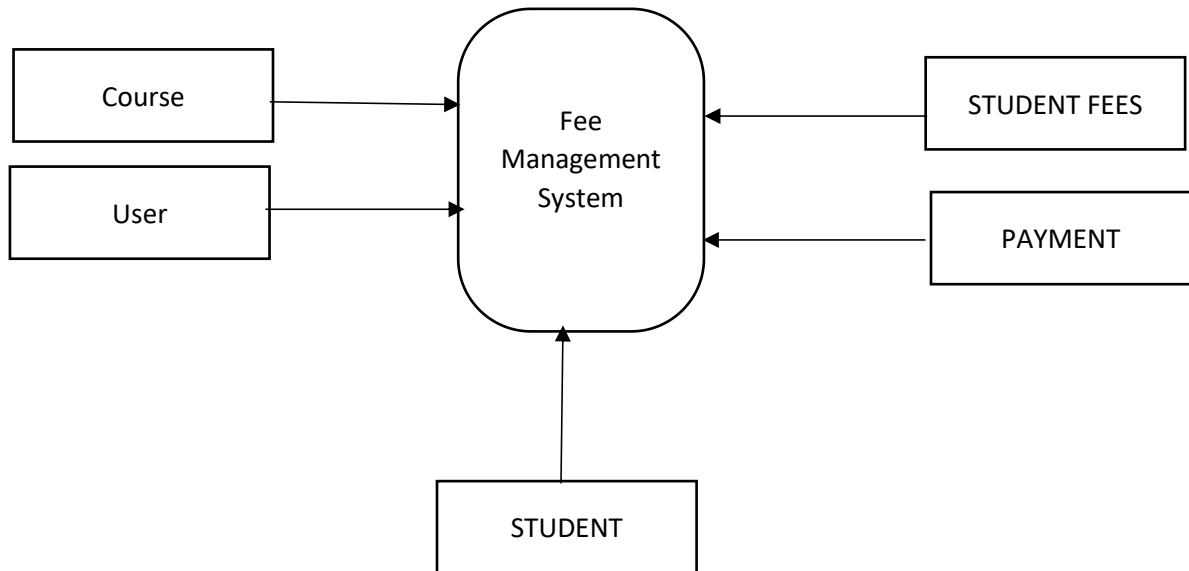


DATA STORE

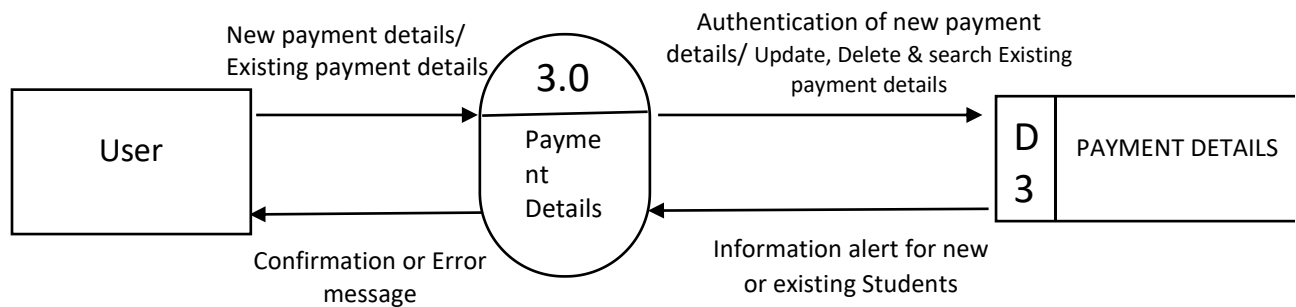
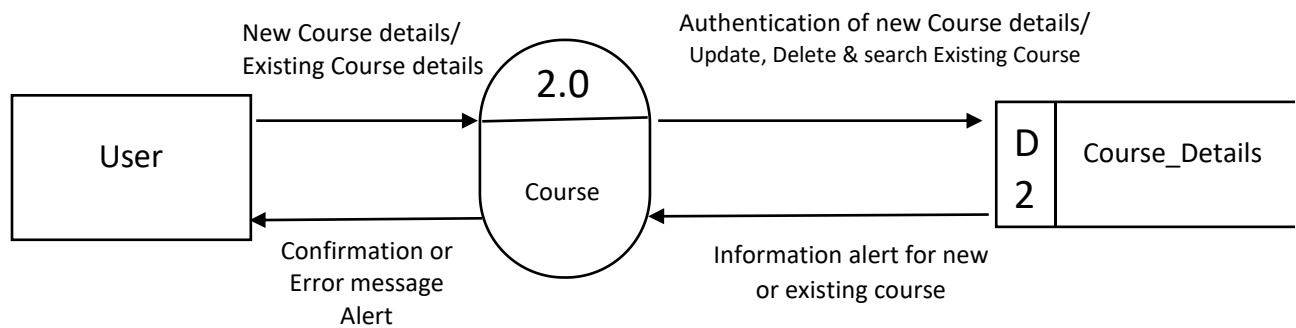
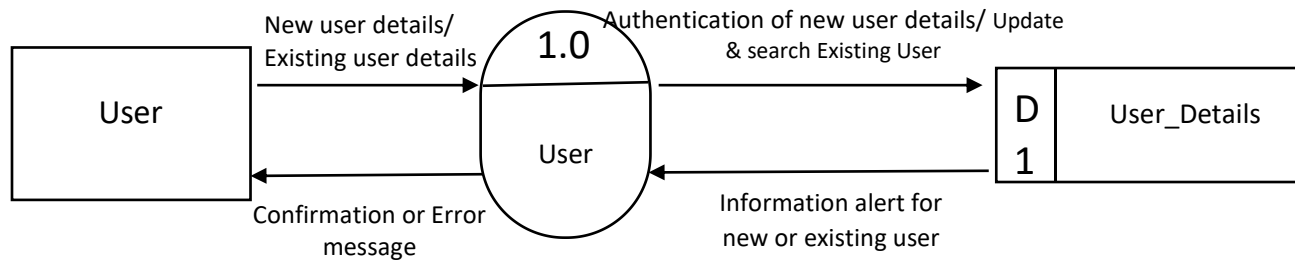
A data store is a holding place for information within the system: It is represented by an openended narrow rectangle. Data stores may be long-term files such as sales ledgers, or may be short-term accumulations: for example, batches of documents that are waiting to be processed. Each data store should be given a reference followed by an arbitrary number.

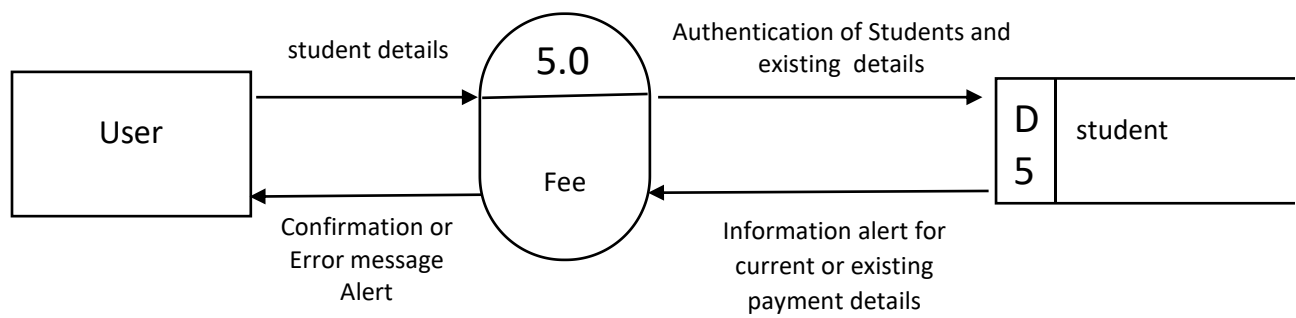
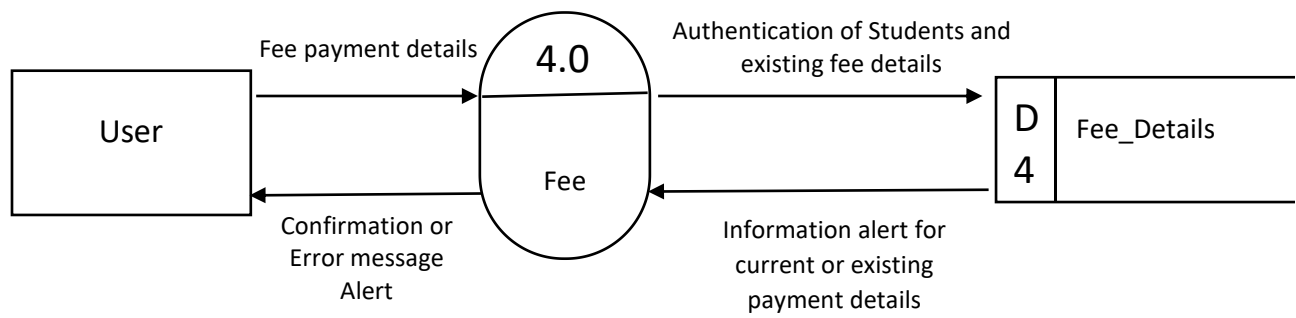


5.1. Context Level DFD

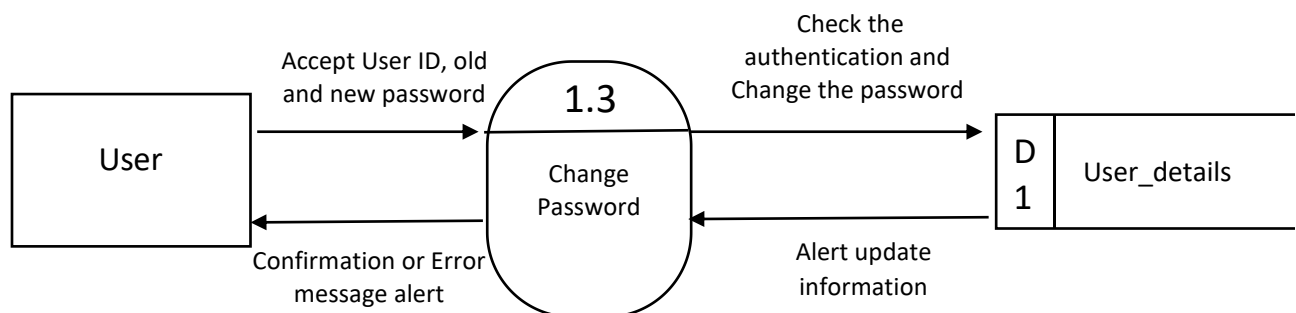
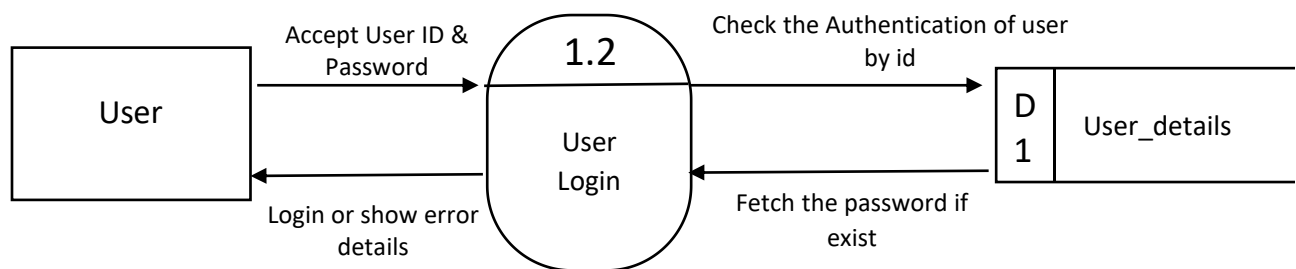
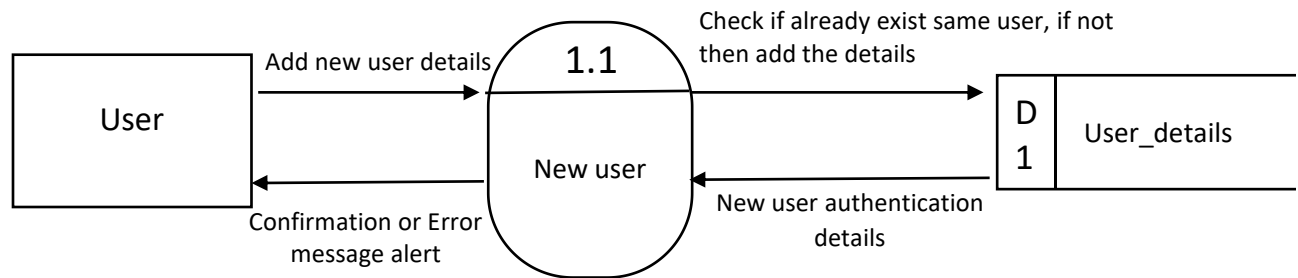


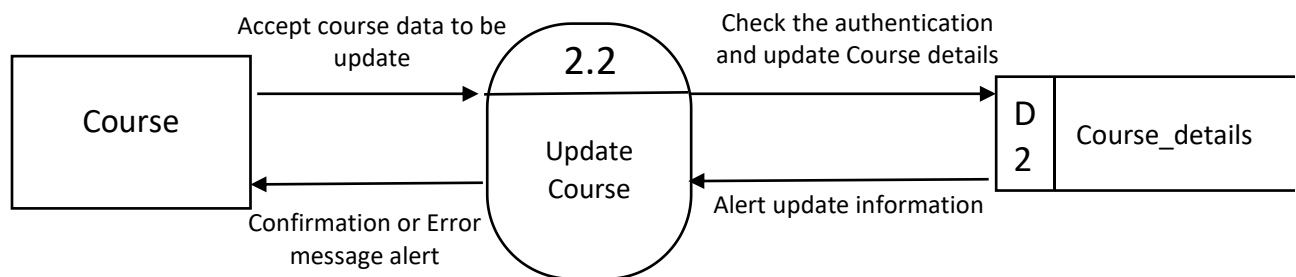
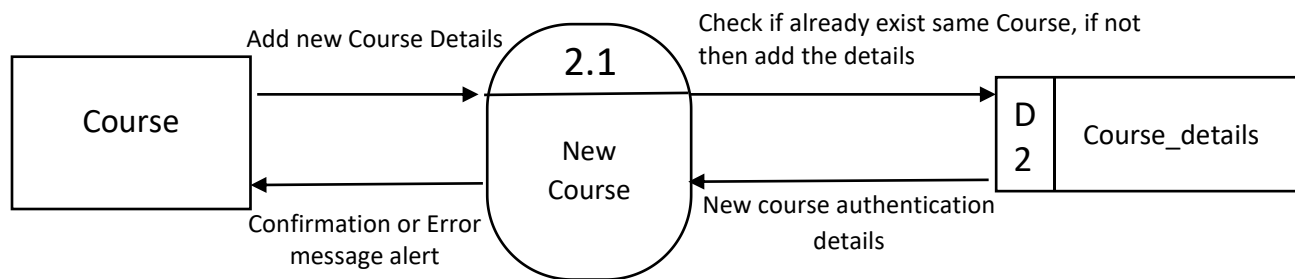
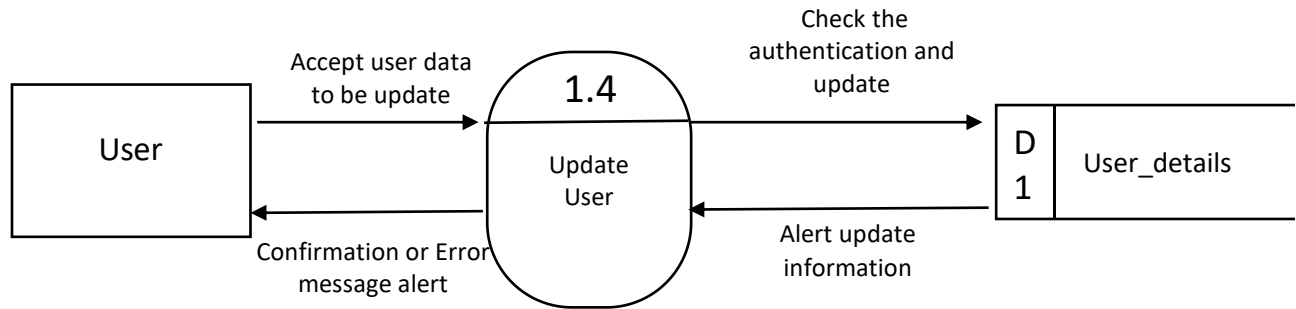
5.2. 1st Level DFD

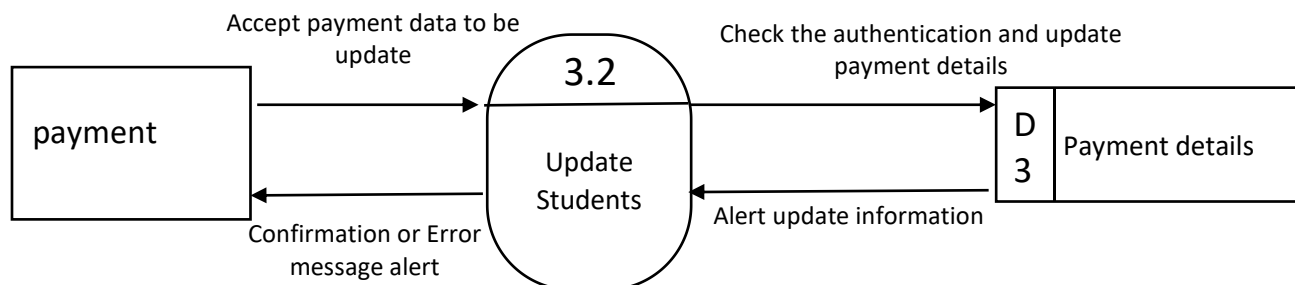
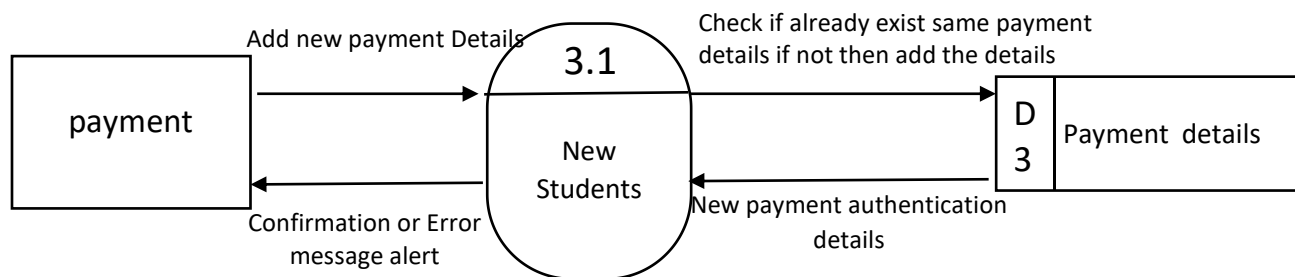
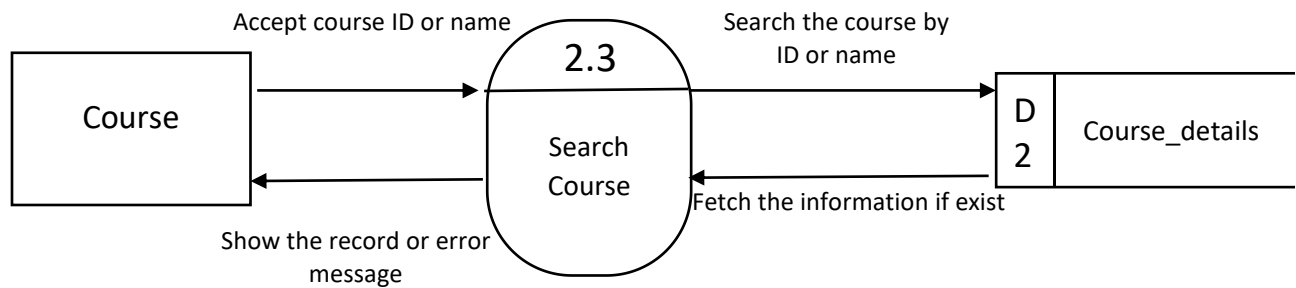


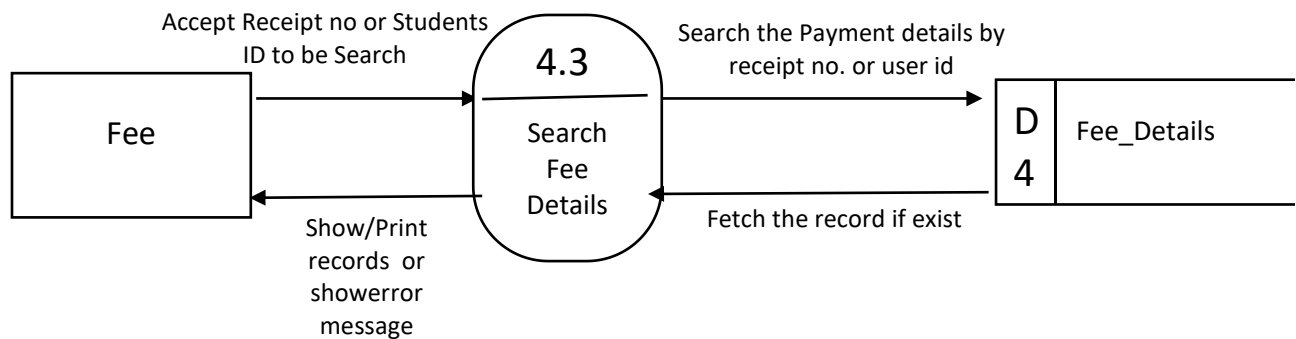
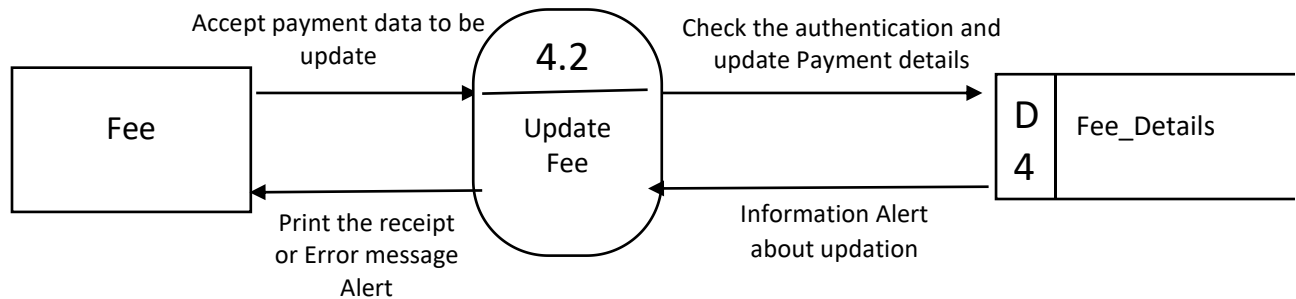
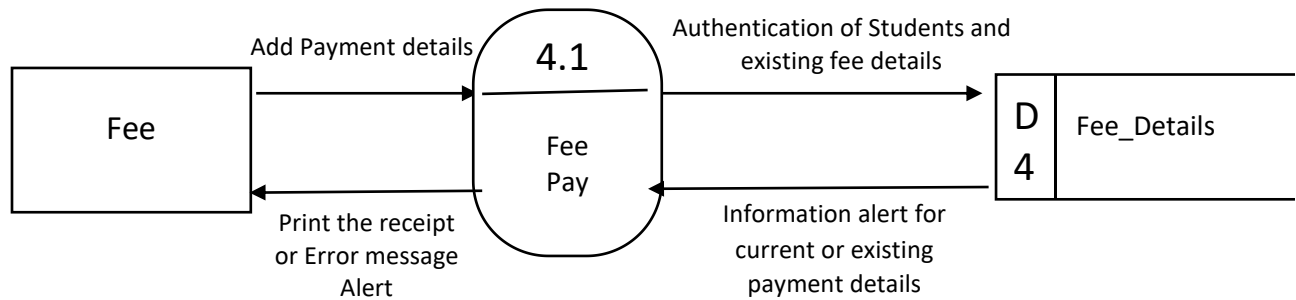
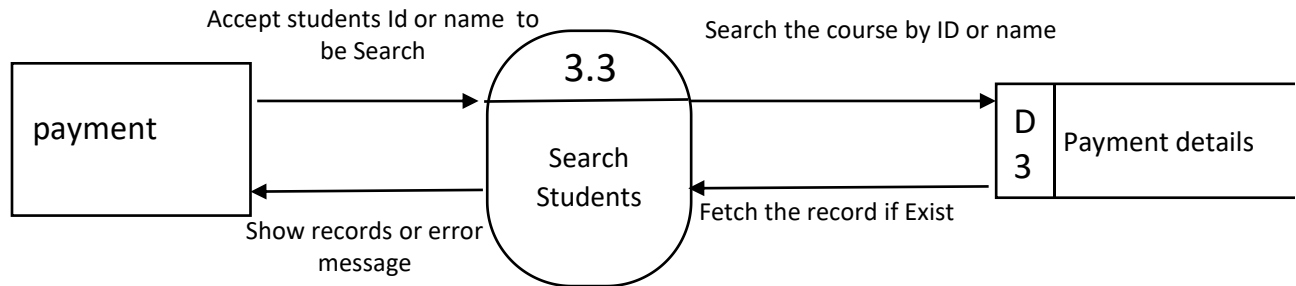


5.3. 2nd Level DFD

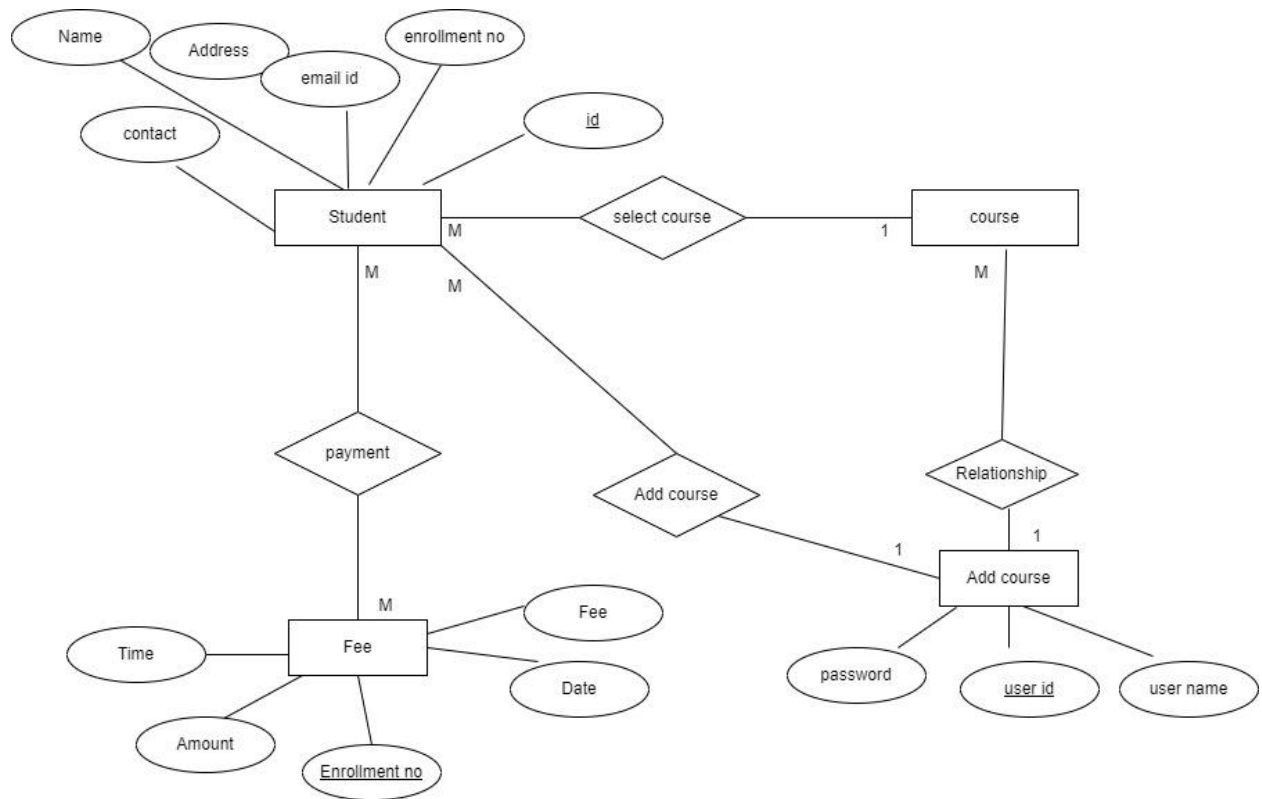








6. ER DIAGRAM



7. DATA DICTIONARY

Admin/User Table structure

DESKTOP-IIDFLGF.f... dbo.login_details			
	user_id	user_name	password
▶	Admin	Rakesh	admin
	Admin2	JAY	123
	Administrator	Karan	jai
*	NULL	NULL	NULL

Course Table Structure

DESKTOP-IIDFLGF.f...bo.course_details						
	course_id	course_name	duration_in_year	no_of_Sem	c_date	c_time
▶	abcd	BCA	3	6	2022-07-10	21:29:22
	nhj	MCA	2	4	2022-07-10	21:31:26
*	NULL	NULL	NULL	NULL	NULL	NULL

Students Table Structure

DESKTOP-IIDFLGF....admission_details											
	en_no	Name	f_name	m_name	gender	email_id	Qualification	passing_year	uni_name	Address	h_no
▶	2022711104432	Priya	priya	priya	Male	priya@gmail.com	bca	2019	Kolhan	Jharkhand	1
	202271111554	Khushboo	Mr. Niram Sahu	Mrs. Kiran DEvi	Female	khushboo@hamil...	Inter	2020	Alkibir	Mango	1
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fee Table Structure

DESKTOP-IIDFLGF.f... - dbo.fee_details						
	re_no	en_no	semester	amount	c_date	c_time
▶	0	0	0	0	2022-07-11	12:00:00
	1	2022711104432	1	40000	2022-07-11	13:36:50
*	NULL	NULL	NULL	NULL	NULL	NULL

8. NORMALIZATION

Normalization is the process of minimizing redundancy from a relation or set of relations. Redundancy in relation may cause insertion, deletion and update anomalies. So, it helps to minimize the redundancy in relations. Normal forms are used to eliminate or reduce redundancy in database tables.

8.1. 0NF

COLUMN NAME	DATA TYPE
ID	Int(30)
Course	varchar(100)
description	text
Level	varchar(150)
Total_amount	float
Date_created	datetime
Fee_id	Int(30)
course_id	int (30)
Description	varchar(200)
Amount	Float
PaymentID	Int(30)
Ef_id	Int(30)
Amount	float
Remarks	text
Date_created	datetime
Student_id	int (30)
Id no	varchar(100)
Name	text
Contact	varchar(100)
Address	text
Email	varchar(200)
date_created	datetime
Id	Int(30)
Student_id	Int(30)
ef no	varchar(200)
Course Id	Int(30)
Total_fee	float
Date_created	datetime
ID	Int(30)
Name	text
username	varchar(200)
Password	text
Type	Tinyint(1)

8.2. 1NF

Course table

COLUMN NAME	DATA TYPE
ID	Int(30)
Course	varchar(100)
description	Text
Level	varchar(150)
Total_amount	Float
Date_created	Datetime

Fee table

COLUMN NAME	DATA TYPE
Fee_id	Int(30)
course_id	int (30)
Description	varchar(200)
Amount	Float

Payment table

COLUMN NAME	DATA TYPE
PaymentID	Int(30)
Ef_id	Int(30)
Amount	float
Remarks	text
Date_created	datetime

Student table

COLUMN NAME	DATA TYPE
Student_id	int (30)
Id_no	varchar(100)
Name	text
Contact	varchar(100)
Address	text
Email	varchar(200)
date_created	datetime

User table

COLUMN NAME	DATA TYPE
ID	Int(30)
Name	text
username	varchar(200)
Password	text
Type	Tinyint(1)

Data Table

COLUMN NAME	DATA TYPE
Id	Int(30)
Student_id	Int(30)
ef_no	varchar(200)
Course Id	Int(30)
Total fee	float
Date_created	datetime

8.3.2NF

Course table

COLUMN NAME	DATA TYPE
ID	Int(30)
Course	varchar(100)
description	Text
Level	varchar(150)
Total_amount	Float
Date_created	Datetime

Fee table

COLUMN NAME	DATA TYPE
Fee_id	Int(30)
course_id	int (30)
Description	varchar(200)
Amount	Float

Payment table

COLUMN NAME	DATA TYPE
PaymentID	Int(30)
Ef_id	Int(30)
Amount	float
Remarks	text
Date_created	datetime

Student table

COLUMN NAME	DATA TYPE
Student_id	int (30)
Id_no	varchar(100)
Name	text
Contact	varchar(100)
Address	text
Email	varchar(200)
date_created	datetime

User table

COLUMN NAME	DATA TYPE
ID	Int(30)
Name	text
username	varchar(200)
Password	text
Type	Tinyint(1)

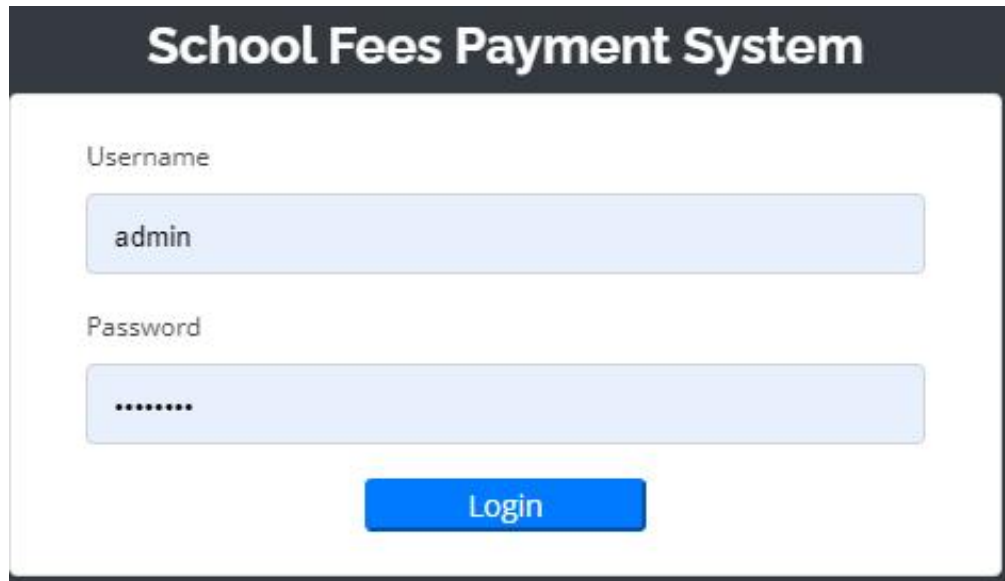
Data Table

COLUMN NAME	DATA TYPE
Id	Int(30)
Student_id	Int(30)
ef_no	varchar(200)
Course_Id	Int(30)
Total_fee	float
Date_created	datetime

Enrollment table

id	Int(30)
Student id	Int(30)
ef_no	varchar(200)
Course Id	Int(30)

10. SCREENSHORTS



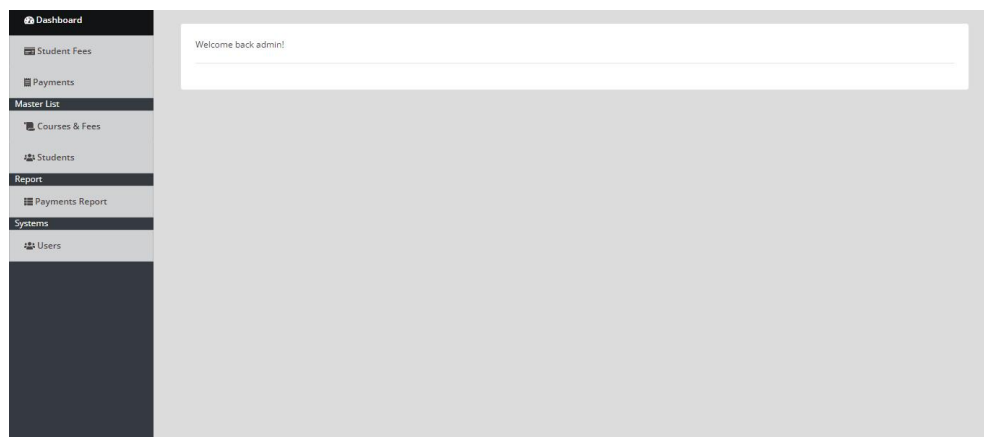
The screenshot shows the login interface of the 'School Fees Payment System'. It features a dark header with the system name. Below the header, there are two input fields: 'Username' with the text 'admin' and 'Password' with masked characters. A blue 'Login' button is positioned below the password field.

School Fees Payment System

Username
admin

Password
.....

Login



Dashboard
Student Fees
Payments
Master List
Courses & Fees
Students
Report
Payments Report
Systems
Users

List of Student fees

+ New

Show 10 entries

Search:

#	ID No.	EF No.	Name	Payable Fee	Paid	Balance	Action
1	1104	1104	ANISHA ROY	10,000.00	0.00	10,000.00	View Edit Delete
2	1103	1103	Rahul Singh	10,000.00	10,000.00	0.00	View Edit Delete
3	1101	1101	ROHIT KUMAR	10,000.00	10,000.00	0.00	View Edit Delete
4	1102	1102	SNEHA KUMARI	10,000.00	10,000.00	0.00	View Edit Delete

Showing 1 to 4 of 4 entries

Previous 1 Next

Dashboard
Student Fees
Payments
Master List
Courses & Fees
Students
Report
Payments Report
Systems
Users

List of Payments

+ New

Show 10 entries

Search:

#	Date	ID No.	EF No.	Name	Paid Amount	Action
1	Aug 24, 2022 23:05 PM	1103	1103	Rahul Singh	10,000.00	View Edit Delete
2	Aug 24, 2022 23:00 PM	1102	1102	SNEHA KUMARI	10,000.00	View Edit Delete
3	Aug 24, 2022 22:39 PM	1101	1101	ROHIT KUMAR	10,000.00	View Edit Delete

Showing 1 to 3 of 3 entries

Previous 1 Next

Dashboard
Student Fees
Payments
Master List
Courses & Fees
Students
Report
Payments Report
Systems
Users

List of Courses and Fees

+ New Entry

Show 10 entries

Search:

#	Course/Level	Description	Total Fee	Action
1	C++ - 1	Basic Of C++.	4,500.00	Edit Delete
2	Java -	Advance Java	10,000.00	Edit Delete
3	Python -	Basic Python.	10,000.00	Edit Delete

Showing 1 to 3 of 3 entries

Previous 1 Next

Dashboard
Student Fees
Payments
Master List
Courses & Fees
Students
Report
Payments Report
Systems
Users

List of Students

+ New

Show 10 entries

Search:

#	ID No.	Name	Information	Action
1	1104	ANISHA ROY	Email: ANISHA@gmail.com Contact #: 675432190 Address: janshedpur, jharkhand	Edit Delete
2	1103	Rahul Singh	Email: rahul@gmail.com Contact #: 765432119 Address: janshedpur jharkhand	Edit Delete
3	1101	ROHIT KUMAR	Email: ROHIT@gmail.com Contact #: 867546321 Address: bihar patna	Edit Delete
4	1102	SNEHA KUMARI	Email: SNEHA@gmail.com Contact #: 623489106 Address: ranchi jharkhand	Edit Delete

Showing 1 to 4 of 4 entries

Previous 1 Next

Dashboard
Student Fees
Payments
Master List
Courses & Fees
Students
Report
Payments Report
Systems
Users

Month August, 2022

#	Date	ID No.	EF No.	Name	Paid Amount	Remarks
1	Aug 24, 2022 11:01 AM	1101	1101	ROHIT KUMAR	10,000.00	
2	Aug 24, 2022 11:02 AM	1102	1102	SNEHA KUMARI	10,000.00	00
3	Aug 24, 2022 11:03 AM	1103	1103	Rahul Singh	10,000.00	00
Total					30,000.00	

Print

Dashboard
Student Fees
Payments
Master List
Courses & Fees
Students
Report
Payments Report
Systems
Users

User List

+ New user

Show 10 entries

Search:

#	Name	Username	Type	Action
1	Admin	admin	Admin	Action

Showing 1 to 1 of 1 entries

Previous 1 Next

11.CODING

```
<!DOCTYPE html>

<html lang="en">


<?php session_start(); ?>

<head>

    <meta charset="utf-8">

    <meta content="width=device-width, initial-scale=1.0" name="viewport">


    <title><?php echo isset($_SESSION['system']['name']) ? $_SESSION['system']['name'] :
" ?></title>


<?php
    if(!isset($_SESSION['login_id']))
        header('location:login.php');
    include('./header.php');
    // include('./auth.php');
?>


</head>

<style>

    body{
        background: #80808045;
    }

    .modal-dialog.large {
        width: 80% !important;
```

```
    max-width: unset;
}

.modal-dialog.mid-large {
    width: 50% !important;
    max-width: unset;
}

#viewer_modal .btn-close {
    position: absolute;
    z-index: 999999;
    /*right: -4.5em;*/
    background: unset;
    color: white;
    border: unset;
    font-size: 27px;
    top: 0;
}

#viewer_modal .modal-dialog {
    width: 80%;
    max-width: unset;
    height: calc(90%);
    max-height: unset;
}

#viewer_modal .modal-content {
    background: black;
    border: unset;
    height: calc(100%);
    display: flex;
    align-items: center;
```

```

        justify-content: center;
    }
    #viewer_modal img,#viewer_modal video{
        max-height: calc(100%);
        max-width: calc(100%);
    }
</style>

<body>
    <?php include 'topbar.php' ?>
    <?php include 'navbar.php' ?>
    <div class="toast" id="alert_toast" role="alert" aria-live="assertive" aria-atomic="true">
        <div class="toast-body text-white">
            </div>
        </div>

    <main id="view-panel" >
        <?php $page = isset($_GET['page']) ? $_GET['page'] : 'home'; ?>
        <?php include $page.'.php' ?>

    </main>

    <div id="preloader"></div>
    <a href="#" class="back-to-top"><i class="icofont-simple-up"></i></a>

    <div class="modal fade" id="confirm_modal" role='dialog'>
        <div class="modal-dialog modal-md" role="document">

```

```

<div class="modal-content">
  <div class="modal-header">
    <h5 class="modal-title">Confirmation</h5>
  </div>
  <div class="modal-body">
    <div id="delete_content"></div>
  </div>
  <div class="modal-footer">
    <button type="button" class="btn btn-primary" id='confirm' onclick="">Continue</button>
    <button type="button" class="btn btn-secondary" data-dismiss="modal">Close</button>
  </div>
window.alert_toast= function($msg = 'TEST',$bg = 'success'){
  $('#alert_toast').removeClass('bg-success')
  $('#alert_toast').removeClass('bg-danger')
  $('#alert_toast').removeClass('bg-info')
  $('#alert_toast').removeClass('bg-warning')

  if($bg == 'success')
    $('#alert_toast').addClass('bg-success')
  if($bg == 'danger')
    $('#alert_toast').addClass('bg-danger')
  if($bg == 'info')
    $('#alert_toast').addClass('bg-info')
  if($bg == 'warning')
    $('#alert_toast').addClass('bg-warning')
  $('#alert_toast .toast-body').html($msg)
  $('#alert_toast').toast({delay:3000}).toast('show');
}

```

```

$(document).ready(function(){
    $('#preloader').fadeOut('fast', function() {
        $(this).remove();
    })
})
$('.datetimepicker').datetimepicker({
    format:'Y/m/d H:i',
    startDate: '+3d'
})
$('.select2').select2({
    placeholder:"Please select here",
    width: "100%"
})
</script>
</html>
Login.php
<!DOCTYPE html>
<html lang="en">
<?php
session_start();
include('./db_connect.php');
ob_start();
// if(!isset($_SESSION['system'])){
    $system = $conn->query("SELECT * FROM system_settings limit 1")->fetch_array();
    foreach($system as $k => $v){
        $_SESSION['system'][$k] = $v;
    }
// }

```

```

ob_end_flush();
?>

        </div>

        <div class="form-group">
            <label for="password" class="control-label">Password</label>
            <input type="password" id="password" name="password" class="form-
control">

        </div>

        <center><button class="btn-sm btn-block btn-wave col-md-4 btn-
primary">Login</button></center>

    </form>

</div>

</div>

</div>

</div>

</main>

```

```

<a href="#" class="back-to-top"><i class="icofont-simple-up"></i></a>

```

```

</body>

```

```

button[type="button"]').attr('disabled',true).html('Logging in...');
    if($(this).find('.alert-danger').length > 0 )
        $(this).find('.alert-danger').remove();
    $.ajax({
        url:'ajax.php?action=login',
        method:'POST',
        data:$(this).serialize(),
        error:err=>{

```

```

        console.log(err)

        $('#login-form button[type="button"]').removeAttr('disabled').html('Login');

    },
    success:function(resp){
        if(resp == 1){
            location.href ='index.php?page=home';
        }else{
            $('#login-form').prepend('<div class="alert alert-danger">Username or password is
incorrect.</div>')
            $('#login-form button[type="button"]').removeAttr('disabled').html('Login');
        }
    }
})
})
</script>
</html>

```

Manage course.php

```

<?php
include 'db_connect.php';
if(isset($_GET['id'])){
    $qry = $conn->query("SELECT * FROM courses where id= ".$_GET['id']);
    foreach($qry->fetch_array() as $k => $val){
        $$k=$val;
    }
}
?>

<div class="container-fluid">

```

```

<form action="" id="manage-course">
  <input type="hidden" name="id" value="<?php echo isset($id) ? $id : " ?>">
  <div class="row">
    <div class="col-lg-6 border-right">
      <h5><b>Course Details</b></h5>
      <hr>
      <div id="msg" class="form-group"></div>
      <div class="form-group">
        <label for="" class="control-label">Course</label>
        <input type="text" class="form-control" name="course" value="<?php echo
isset($course) ? $course : " ?>" required>
      </div>
      <div class="form-group">
        <label for="" class="control-label">Level</label>
        <input type="text" class="form-control" name="level" value="<?php echo
isset($level) ? $level : " ?>" required>
      </div>
      <div class="form-group">
        <label for="" class="control-label">Description</label>
        <textarea name="description" id="" cols="30" rows="4" class="form-control"
required=""><?php echo isset($description) ? $description : " ?></textarea>
      </div>
    </div>
    <div class="col-lg-6">
      <h5><b>Fee Details</b></h5>
      <hr>
      <div class="row">
        <div class="form-group">
          <label for="ft" class="control-label">Fee Type</label>

```

```

        <input type="text" id="ft" class="form-control-sm">
    </div>

    <div class="form-group">
        <label for="" class="control-label">Amount</label>
        <input type="number" step="any" min="0" id="amount" class="form-control-sm
text-right">

</tbody>
<tfoot>
    <tr>
        <th colspan="2" class="text-center">Total</th>
        <th class="text-right">
            <input type="hidden" name="total_amount" value="<?php echo isset($total) ?
$total : 0 ?>">
            <span class="tamount"><?php echo isset($total) ? number_format($total,2) :
'0.00' ?></span>
        </th>
    </tr>
</tfoot>
</table>
</div>
</div>
</form>
</div>
<div id="fee_clone" style="display: none">
    <table >
        <tr>
            <td class="text-center"><button class="btn-sm btn-outline-danger" type="button"
onclick="rem_list($(this))" ><i class="fa fa-times"></i></button></td>

```

```

        <td>
            <input type="hidden" name="fid[]">
            <input type="hidden" name="type[]">
            <p><small><b class="ftype"></b></small></p>
        </td>
        <td>
            <input type="hidden" name="amount[]">
            <p class="text-right"><small><b class="famount"></b></small></p>
        </td>
    </tr>
</table>
</div>
<script>
    $('#manage-course').on('reset',function(){
        $('#msg').html("")
        $('input:hidden').val("")
    })
    $('#add_fee').click(function(){
        var ft = $('#ft').val()
        var amount = $('#amount').val()
        if(amount == "" || ft == ""){
            alert_toast("Please fill the Fee Type and Amount field first.", 'warning')
            location.reload()
        },1000)
        }else if(resp == 2){
            $('#msg').html('<div class="alert alert-danger mx-2">Course Name & Level already
exist.</div>')
            end_load()
        }
    }

```

```

        }
    })
})

$('.select2').select2({
    placeholder:"Please Select here",
    width:'100%'
})
</script>
Manage fee.php
<?php include 'db_connect.php' ?>
<?php
if(isset($_GET['id'])){
    $qry = $conn->query("SELECT * FROM student_ef_list where id = {$_GET['id']} ");
    foreach($qry->fetch_array() as $k => $v){
        $$k = $v;
    }
}
?>
<div class="container-fluid">
    <form id="manage-fees">
        <div id="msg"></div>
        <input type="hidden" name="id" value="<?php echo isset($id) ? $id : " ?>">
        <div class="form-group">
            <label for="" class="control-label">Enrollment No./ E.F. No.</label>
            <input type="text" class="form-control" name="ef_no" value="<?php echo
isset($ef_no) ? $ef_no : " ?>" required>
        </div>
        <div class="form-group">

```

```

<label for="" class="control-label">Student</label>

<select name="student_id" id="student_id" class="custom-select input-sm select2">

    <option value=""></option>

    <?php
        $student = $conn->query("SELECT * FROM student order by name asc ");

method:'POST',
data:$(this).serialize(),
error:err=>{
    console.log(err)
    end_load()
},
success:function(resp){
    if(resp == 1){
        location.reload();
        alert_toast("Data successfully saved.",'success')
        setTimeout(function(){
            location.reload()
        },1000)
    }else if(resp == 2){
        $('#msg').html('<div class="alert alert-danger">EF Nunmber already exist.</div>')
        end_load()
    }
}
})
})
</script>
Manage payment.php

```

```

<?php include 'db_connect.php' ?>

<?php
if(isset($_GET['id'])){
    $qry = $conn->query("SELECT * FROM payments where id = {$_GET['id']} ");
    foreach($qry->fetch_array() as $k => $v){
        $$k = $v;
    }
}
?>

<div class="container-fluid">
    <form id="manage-payment">
        <div id="msg"></div>
        <input type="hidden" name="id" value="<?php echo isset($id) ? $id : " ?>">
        <div class="form-group">
            <label for="" class="control-label">EF.NO./Student</label>
            <select name="ef_id" id="ef_id" class="custom-select input-sm select2">
                <option value=""></option>
                <?php
                    $fees = $conn->query("SELECT ef.*,s.name as sname,s.id_no FROM
student_ef_list ef inner join student s on s.id = ef.student_id order by s.name asc ");
                    while($row= $fees->fetch_assoc()):
                        data:$(this).serialize(),
                        error:err=>{
                            console.log(err)
                            end_load()
                        },
                        success:function(resp){
                            resp = JSON.parse(resp)
                            if(resp.status == 1){

```

```

        alert_toast("Data successfully saved.", 'success')

        setTimeout(function(){

            var nw =
window.open('receipt.php?ef_id='+resp.ef_id+'&pid='+resp.pid,"_blank","width=900,height=60
0")

            setTimeout(function(){

                nw.print()

                setTimeout(function(){

                    nw.close()

                    location.reload()

                },500)

            },500)

        },500)

    }

}

})

})

</script>

```

Types Of Testing

The system was designed according to the requirement of the system. But we are not 100% confident. The lack of confidence stems from several things. First the system deals with large number of states, complex logic and activities. So some error might occur in the system. Error may be software, which is known as “SOFTWARE ERROR” i.e. the software doesn’t do what the requirement says. So an exhaustive and thorough testing must be conducted to ascertain. Whether the system produces right results. The project guide and the user both did testing

Module Testing

The testing was done in several stages. First each program module was tested as a single program, which is also known as module testing or unit testing. In unit testing a set of data as input was given to the module and observed what output data is produced. In addition, the logic and boundary condition for input and output data was also checked. The interface between this module and others was checked for correctness. While collecting the input data for testing the program module it was kept in mind that input should be from all classes, so the entire condition of the program could also be checked.

Integrating testing

When the individual program modules were working properly, we combine the module in the working system. This integration is planned and coordinated so that when an error occurs, we have an idea of what caused it. Integration testing is the process of verifying that the components of a system work together as described in the program design specification. For testing, the system was viewed as a hierarchy of modules. We began with the module at the highest level of design and worked down. The next modules to be tested are those that call previously tested modules.

Function Testing

Once we are sure that information is passed among modules according to the design prescription we tested the system to assure whether the function described in the requirement specification are actually performed by the integrated system.

Acceptance Test

When the function test completes then we involved the user to make sure that system works according to the user's expectation. Thus, the user did the acceptance test.

Implementation

Once the system was tested (module wise as well as integrated) satisfactorily, and then comes the implementation of the system. Implementation is the process of changing from old system (manual) to the new system (computerized). Some training was also given to the user about how to work on the new system and finally the system was successfully adopted.

5.3. VALIDATION CHECK

The basic objective of the requirement validation activity is to ensure that "Inventory Management System" reflects the actual requirements accurately and clearly. Checking data for appropriateness and accuracy is called validating. Data entry checks can be built into the database packages to prevent invalid data from being entered. For instance, in database, format checking (determining if the data are in the correct format) is done automatically because at the time of creating a database file. Range checking (determining if the data fall within an acceptable range) and accuracy checking (making sure that an entry is possible) can be built by writing a simple program in database.

12. IMPLEMENTATION AND SYSTEM TESTING

The aim of the system testing process was to determine all defects in our project .The program was subjected to a set of test inputs and various observations were made and based on these observations it will be decided whether the program behaves as expected or not. Our Project went through two levels of testing

1. Unit testing
2. integration testing

12.1. Unit testing

Unit testing is undertaken when a module has been created and successfully reviewed .In order to test a single module we need to provide a complete environment i.e. besides the module we would require

- The procedures belonging to other modules that the module under test calls
- Non local data structures that module accesses
- A procedure to call the functions of the module under test with appropriate parameters Unit testing was done on each and every module that is described under module description of chapter

12.1.1. Test for the admin module

Testing admin login form-This form is used for log in of administrator of the system. In this we enter the username and password if both are correct administration page will open otherwise if any of data is wrong it will get redirected back to the login page and again ask for username and password

Student account addition- In this section the admin can verify student details from student academic info and then only add student details to main library database it contains add and delete buttons if user click add button data will be added to student database and if he clicks delete button the student data will be deleted

Course Addition- Admin can enter details of Course and can add the details to the main Course table.

12.2. Integration testing

In this type of testing we test various integration of the project module by providing the input. The primary objective is to test the module interfaces in order to ensure that no errors are occurring when one module invokes the other module

13.CONCLUSION

This software provides a computerized version of fee management system which will benefit the staff of the fee department. It makes entire process computerize where staff can search records and can generate reports and do fee transactions.